



Aula de MySQL

Banco de Dados — 2ª Aula

Desenvolvimento de Sistemas

Conteúdo desta aula:

Criar um Banco de Dados e Tabelas

Inserir Dados (INSERT INTO)

Consultas com SELECT e WHERE

1. Criando o Banco de Dados e as Tabelas

Antes de armazenar qualquer informação, precisamos criar nosso banco de dados e definir sua estrutura através das tabelas.

1.1 Criando o Banco de Dados

O comando CREATE DATABASE cria um novo banco. Em seguida, USE seleciona qual banco iremos utilizar:

```
-- Criar o banco de dados CREATE DATABASE escola; -- Selecionar o banco para uso USE escola;
```

💡 Sempre execute USE antes de criar tabelas ou inserir dados, senão o MySQL não saberá em qual banco trabalhar.

1.2 Criando as Tabelas

Para esta aula, vamos criar um sistema simples de escola com alunos e cursos:

```
-- Tabela de cursos CREATE TABLE cursos ( id INT AUTO_INCREMENT PRIMARY KEY, nome VARCHAR(100) NOT NULL, duracao INT, -- duração em meses turno VARCHAR(10) -- Manhã, Tarde ou Noite ); -- Tabela de alunos CREATE TABLE alunos ( id INT AUTO_INCREMENT PRIMARY KEY, nome VARCHAR(100) NOT NULL, email VARCHAR(150), idade INT, cidade VARCHAR(80), curso_id INT, ativo BOOLEAN DEFAULT TRUE );
```

Principais tipos de dados utilizados:

Tipo	Uso	Exemplo
INT	Números inteiros	idade, id, duracao
VARCHAR (n)	Textos com limite de n chars	nome, email, cidade
BOOLEAN	Verdadeiro ou Falso	ativo (TRUE/FALSE)
AUTO_INCREMENT	Número gerado automaticamente	id incrementa sozinho

⚠️ PRIMARY KEY garante que cada registro seja único. Nunca remova ela de suas tabelas!

2. Inserindo Dados (INSERT INTO)

Com a estrutura criada, podemos popular as tabelas com dados usando o comando INSERT INTO.

2.1 Inserindo um registro por vez

```
-- Inserir um curso INSERT INTO cursos (nome, duracao, turno) VALUES  
( 'Desenvolvimento de Sistemas', 36, 'Manhã'); -- Inserir outro curso INSERT  
INTO cursos (nome, duracao, turno) VALUES ( 'Administração', 24, 'Noite');
```

2.2 Inserindo vários registros de uma vez

É possível inserir múltiplos registros em um único comando, separando cada grupo de valores por vírgula:

```
-- Inserir vários cursos de uma vez INSERT INTO cursos (nome, duracao,  
turno) VALUES      ('Enfermagem', 36, 'Tarde'),      ('Contabilidade', 24,  
'Manhã'),      ('Informática', 18, 'Noite'); -- Inserir alunos INSERT INTO  
alunos (nome, email, idade, cidade, curso_id, ativo) VALUES      ('Ana  
Silva',      'ana@email.com', 20, 'São Paulo', 1, TRUE),      ('Bruno  
Souza',      'bruno@email.com', 22, 'Campinas', 1, TRUE),      ('Carla  
Pereira', 'carla@email.com', 19, 'São Paulo', 2, TRUE),      ('Diego  
Martins', 'diego@email.com', 25, 'Santos', 3, FALSE),      ('Elena  
Costa',      'elena@email.com', 21, 'Campinas', 1, TRUE),      ('Felipe  
Ramos',      'felipe@email.com', 18, 'São Paulo', 4, TRUE),      ('Gabi Alves',  
'gabi@email.com', 23, 'Sorocaba', 2, FALSE),      ('Hugo Lima',  
'hugo@email.com', 20, 'Campinas', 5, TRUE);
```

 Inserir múltiplos registros em um único INSERT é mais eficiente do que vários INSERTs separados.

3. Consultando Dados com SELECT e WHERE

O SELECT é o comando mais utilizado no MySQL. Ele busca dados nas tabelas. O WHERE permite filtrar os resultados.

3.1 SELECT básico (sem filtro)

```
-- Ver todos os alunos SELECT * FROM alunos; -- Ver apenas nome e cidade
SELECT nome, cidade FROM alunos;
```

💡 O * significa "todas as colunas". Em produção, prefira listar as colunas que realmente precisa.

3.2 Filtrando com WHERE — Igualdade

O WHERE filtra registros que atendem a uma condição. O operador = verifica igualdade:

```
-- Alunos de São Paulo SELECT * FROM alunos WHERE cidade = 'São Paulo'; --
Alunos do curso_id 1 (Desenvolvimento de Sistemas) SELECT nome, email FROM
alunos WHERE curso_id = 1;
```

3.3 Filtrando com WHERE — Operadores de Comparação

Podemos comparar valores numéricos usando os operadores maior, menor, maior ou igual e menor ou igual:

```
-- Alunos com idade maior que 20 SELECT nome, idade FROM alunos WHERE idade
> 20; -- Alunos com 20 anos ou menos SELECT nome, idade FROM alunos WHERE
idade <= 20; -- Alunos com exatamente 22 anos SELECT * FROM alunos WHERE
idade = 22;
```

3.4 Filtrando com WHERE — Diferente (!= ou <>)

```
-- Alunos que NÃO são de São Paulo SELECT nome, cidade FROM alunos WHERE
cidade != 'São Paulo'; -- Equivalente usando <> SELECT nome, cidade FROM
alunos WHERE cidade <> 'São Paulo';
```

3.5 Filtrando com WHERE — AND e OR

Combinamos condições com AND (ambas devem ser verdadeiras) ou OR (pelo menos uma deve ser verdadeira):

```
-- Alunos de São Paulo E maiores de 19 anos SELECT nome, cidade, idade FROM
alunos WHERE cidade = 'São Paulo' AND idade > 19; -- Alunos de Campinas OU
Sorocaba SELECT nome, cidade FROM alunos WHERE cidade = 'Campinas' OR cidade
= 'Sorocaba';
```

💡 Use parênteses quando combinar AND e OR para garantir a ordem correta das condições.

3.6 Filtrando com WHERE — LIKE (busca parcial)

O LIKE permite buscar registros que contenham, comecem ou terminem com determinado texto. O % representa qualquer sequência de caracteres:

```
-- Alunos cujo nome começa com 'A' SELECT * FROM alunos WHERE nome LIKE
'A%'; -- Alunos cujo nome termina com 'a' SELECT * FROM alunos WHERE nome
LIKE '%a'; -- Alunos cujo email contém 'email.com' SELECT nome, email FROM
alunos WHERE email LIKE '%email.com';
```

3.7 Filtrando com WHERE — IN (lista de valores)

O IN verifica se o valor está dentro de uma lista, evitando vários OR:

```
-- Alunos de São Paulo, Campinas ou Santos SELECT nome, cidade FROM alunos
WHERE cidade IN ('São Paulo', 'Campinas', 'Santos'); -- Alunos dos cursos
1, 2 ou 3 SELECT nome, curso_id FROM alunos WHERE curso_id IN (1, 2, 3);
```

3.8 Filtrando com WHERE — BETWEEN (intervalo)

O BETWEEN filtra valores dentro de um intervalo (inclusivo nos dois extremos):

```
-- Alunos com idade entre 19 e 22 anos SELECT nome, idade FROM alunos WHERE
idade BETWEEN 19 AND 22;
```

3.9 Filtrando com WHERE — IS NULL / IS NOT NULL

Para verificar campos sem valor (NULL) ou com valor preenchido:

```
-- Alunos sem e-mail cadastrado SELECT nome FROM alunos WHERE email IS NULL;
-- Alunos com e-mail cadastrado SELECT nome, email FROM alunos WHERE email
IS NOT NULL;
```

 *Nunca use = NULL para comparar nulos. O correto sempre é IS NULL ou IS NOT NULL.*

3.10 Filtrando alunos ativos ou inativos

```
-- Alunos ativos SELECT nome FROM alunos WHERE ativo = TRUE; -- Alunos
inativos SELECT nome FROM alunos WHERE ativo = FALSE;
```

4. Exercícios Práticos

Com base nas tabelas `escola.alunos` e `escola.cursos` criadas nesta aula, escreva as queries para:

- Buscar todos os alunos da cidade de Campinas.
- Listar o nome e email dos alunos com idade maior ou igual a 21 anos.
- Encontrar os alunos matriculados no curso_id 1 que estão ativos.
- Listar alunos cujo nome começa com a letra 'E'.
- Buscar alunos que NÃO são de São Paulo e têm menos de 25 anos.
- Listar todos os alunos com idade entre 18 e 20 anos.
- Mostrar os alunos das cidades São Paulo ou Santos que estão inativos.

5. Próximos Passos Sugeridos

Parabéns por chegar até aqui! Vocês já sabem criar banco, inserir e consultar dados. A próxima etapa natural da jornada é:

Recomendação: JOIN — Unindo Tabelas

Até agora consultamos uma tabela por vez. Na próxima aula, aprendemos a relacionar tabelas com o JOIN, que é o recurso mais poderoso do SQL relacional.

Exemplo do que vocês conseguirão fazer:

```
-- Mostrar o nome do aluno junto com o nome do seu curso SELECT alunos.nome,  
cursos.nome AS curso FROM alunos JOIN cursos ON alunos.curso_id = cursos.id;  
-- Com WHERE + JOIN: alunos ativos e seu curso SELECT alunos.nome,  
cursos.nome AS curso, alunos.cidade FROM alunos JOIN cursos ON  
alunos.curso_id = cursos.id WHERE alunos.ativo = TRUE;
```

 *O JOIN vai unir tudo que vocês aprenderam até aqui. Dominar o JOIN é o divisor de águas entre quem usa SQL de forma básica e quem realmente sabe trabalhar com banco de dados.*

Outros conteúdos que virão na sequência:

- UPDATE — Atualizar registros existentes
- DELETE — Remover registros da base
- ORDER BY e LIMIT — Ordenar e paginar resultados
- GROUP BY e funções de agregação (COUNT, SUM, AVG)
- Chaves Estrangeiras (FOREIGN KEY) e integridade referencial

Bons estudos! 

MySQL — Aula 2 — Desenvolvimento de Sistemas